

Pool loss of flow: STREAM.jl against Python STREAM

2.8 MW, 40 high channels, 10 low channels, pump trip at $t = 0$, run to 300 s. Every input is a placeholder and describes no plant.

	Julia	Python	Python – Julia
steady primary flow [kg/s]	16.189	16.189	-0.00%
steady high flow per channel [kg/s]	0.35489	0.35489	-0.00%
steady high outlet temperature [°C]	77.419	77.419	-1.06e-06
steady low flow per channel [kg/s]	0.19928	0.19928	-0.00%
steady low outlet temperature [°C]	68.591	68.591	-8.57e-07
steady high heat transfer coefficient, hottest cell [W/(m ² ·K)]	16660	16669	+0.05%
steady high pressure drop, inlet to outlet [kPa]	0.88453	0.88453	-0.00%
steady high friction pressure drop [kPa]	6.6683	6.6683	-0.00%
steady low heat transfer coefficient, hottest cell [W/(m ² ·K)]	9770.3	9770.3	-0.00%
steady low pressure drop, inlet to outlet [kPa]	-3.3333	-3.3333	-0.00%
steady low friction pressure drop [kPa]	2.465	2.465	-0.00%
low-flow trip [s]	1.794	1.864	+0.07
primary flow at each code's trip, share of design	0.85	0.84688	-0.00312
flapper opens [s]	97.181	97.208	+0.0271
high turns around [s]	101.2	101.3	+0.1
low turns around [s]	106.4	106.4	+0
high peak coolant, rods in to $t = 300$ s [°C]	85.262	85.28	+0.0174
high peak wall, rods in to $t = 300$ s [°C]	120.6	120.63	+0.0307
high peak plate, rods in to $t = 300$ s [°C]	120.67	120.7	+0.0305
low peak coolant, rods in to $t = 300$ s [°C]	75.332	75.388	+0.056
low peak wall, rods in to $t = 300$ s [°C]	90.855	90.789	-0.0659
low peak plate, rods in to $t = 300$ s [°C]	90.878	90.813	-0.0659
total power [share] at $t = 300$ s	0.026908	0.026909	+0.00%
high flow per channel [kg/s] at $t = 300$ s	-0.012817	-0.012808	+0.07%
low flow per channel [kg/s] at $t = 300$ s	-0.0072575	-0.0072484	+0.13%
largest error, primary flow [kg/s], at $t = 0.1$ s	16.034	16.073	+0.24%
largest error, total power [share], at $t = 2.0$ s	0.38337	0.66422	+73.26%
largest error, high flow [kg/s], at $t = 97.6$ s	0.024398	0.024973	+2.36%
largest error, low flow [kg/s], at $t = 97.4$ s	0.01668	0.017182	+3.01%
largest error, high hottest coolant [°C], at $t = 2.2$ s	70.54	76.077	+5.54
largest error, high hottest plate [°C], at $t = 2.0$ s	122.61	137.02	+14.4
largest error, low hottest coolant [°C], at $t = 2.4$ s	62.208	64.951	+2.74
largest error, low hottest plate [°C], at $t = 2.0$ s	105.35	113.28	+7.93

Known differences between the builds

Low-flow trip

Julia: Callback finds the crossing of the primary flow

Python: State machine reads the primary flow, as the kinetics' reference flow, between solver steps at most 0.05 s apart

Effect: Python trips up to 0.05 s after its flow crosses the setpoint, and the pump ramp makes that crossing a little later, so power and plate temperatures differ sharply while the rods go in

Flapper opening

Julia: Callback finds the exact crossing

Python: Checked between solver steps at most 0.05 s apart

Effect: Python opens up to 0.05 s late, and flows differ sharply until both are open

Equation handling

Julia: ModelingToolkit reduces the index, then Rodas5P

Python: IDA on the unreduced system, which is above index 1, with the algebraic variables kept out of its error test and its step capped at 0.05 s

Effect: Without both, IDA fails within a second of the flapper opening

Pump trip

Julia: Head steps to zero at $t = 0$

Python: Head ramps to zero over 50 ms, since IDA's initial-condition solver fails on the step

Effect: The primary flow runs a fraction of a percent high just after the trip

Saved points

Julia: The solver steps onto every saved time

Python: IDA interpolates between its own steps

Effect: None beyond interpolation

The next page counts the unknowns and times each phase. Each page after it shows one quantity in three time windows, with Python minus Julia underneath.

Problem size and wall time

What each solver integrates for this case, and how long each phase takes.

unknowns	Julia	Python
unknowns in the model as built	892	234
unknowns the solver integrates	162	234
of which differential	120	122
of which algebraic	42	112
observed, computed after the solve	756	-

Julia's algebraic unknowns	count
flapper.outlet.p	1
high_weight_out.outlet.p	1
high.ch.thermal_left.T	10
high.ch.thermal_right.T	10
low.ch.thermal_left.T	10
low.ch.thermal_right.T	10

Python's algebraic unknowns	count	in
Tin	12	12 components
pressure	12	12 components
h_left	20	ch_high, ch_low
h_right	20	ch_high, ch_low
T_wall_left	20	fuel_high, fuel_low
T_wall_right	20	fuel_high, fuel_low
power	1	pk
p_abs	2	Kirchhoff
mdot2	5	Kirchhoff

wall time [s]	cold	Julia		Python	
		same model	rebuilt	first pass	second pass
load packages	9.8	-	-	3.3	-
build and compile	31.8 (65%)	-	12.5	0.1	0.0
steady state	121.0 (99%)	1.0	0.7	3.0	0.3
transient	53.0 (66%)	16.8	19.1	222.3	211.7

ModelingToolkit removes aliases and solves every equation it can rearrange explicitly before the solve, so Julia integrates 162 of the 892 unknowns it builds and recomputes the other 756 from them afterwards. 42 of those stay algebraic: the plate surface temperatures, one per face and cell, where $h(T_{\text{wall}}) \cdot (T_{\text{wall}} - T)$ meets conduction in the plate and h itself depends on T_{wall} , and the pressures at the two junctions where branches meet, the lower plenum and the flapper node.

Rodas5P integrates $M \cdot u' = f(u)$ with a zero on each algebraic row of M . It is a Rosenbrock method built for index-1 systems in that form, so a step costs a few linear solves and no Newton iterations.

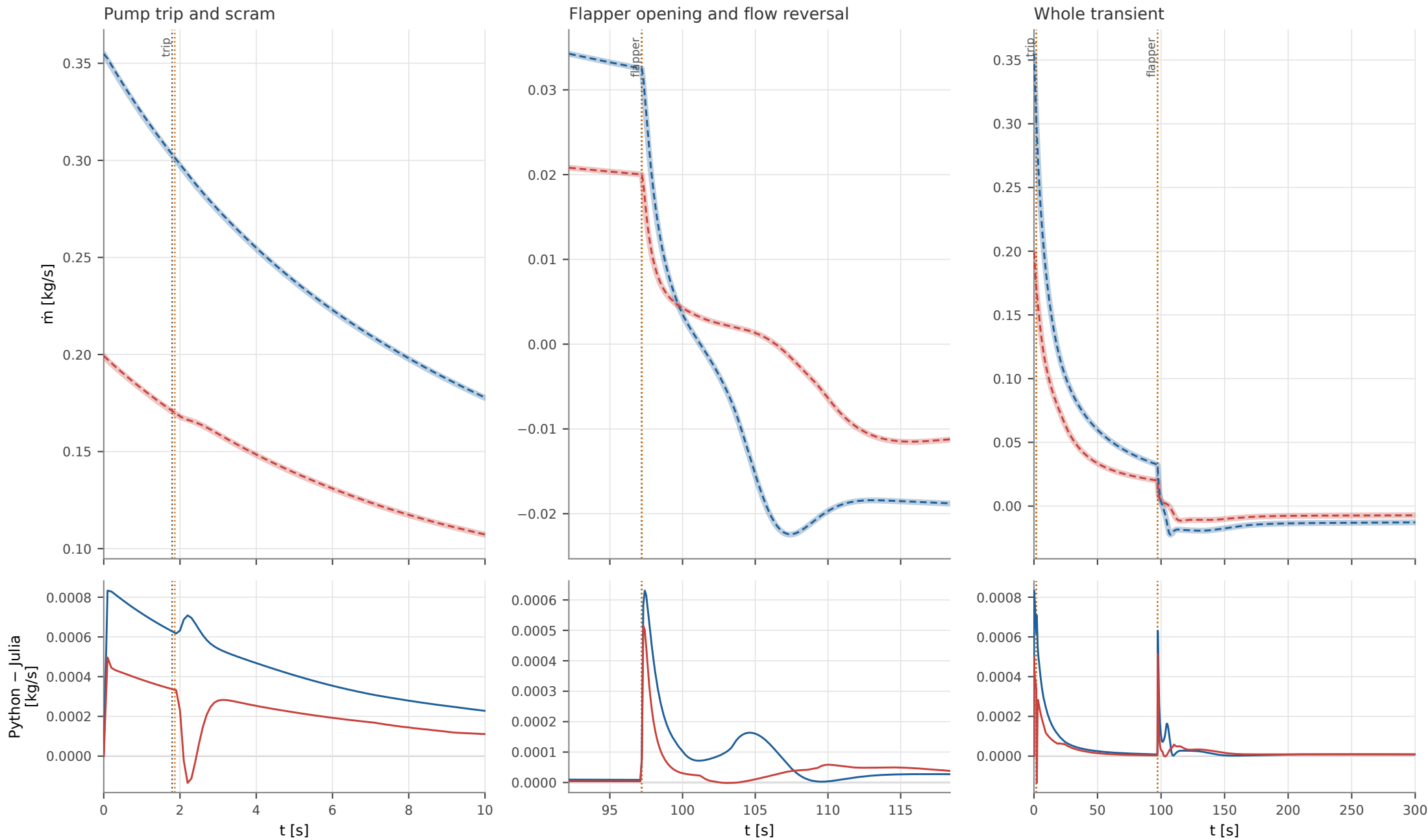
Python hands IDA all 234 unknowns. 112 of them are algebraic: the heat transfer coefficient as well as the wall temperature of every face and cell, each component's inlet temperature and pressure change, and the flow derivatives of its Kirchhoff formulation, which put the system above index 1.

Julia's cold column includes compiling, with its share in brackets. The same-model column solves the built model again, and the rebuilt column builds it again in the same process, which compiles little. Python's second pass repeats the first in the same process.

Flow per channel

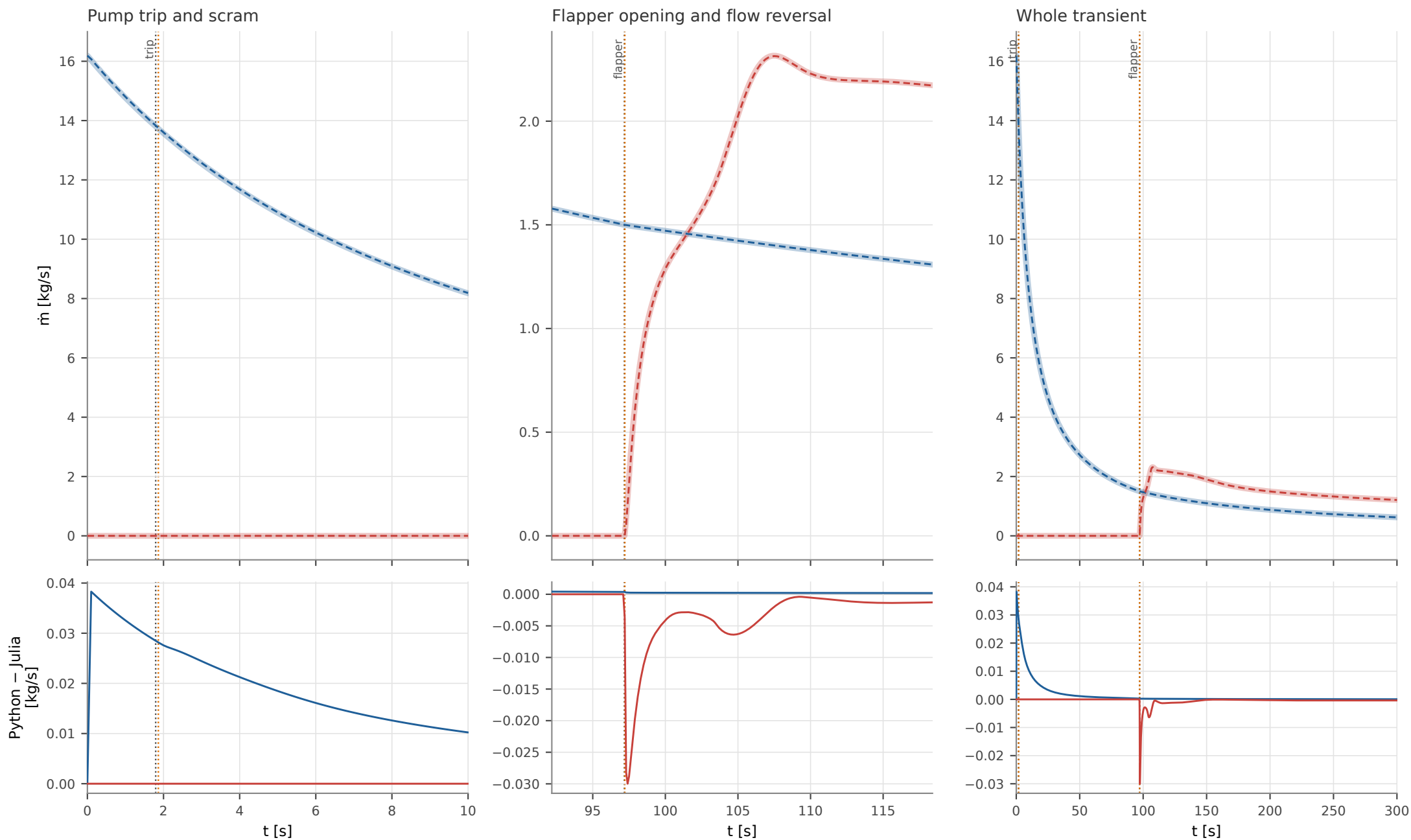
Positive is the forced, downward direction; natural circulation runs upward.

high channel low channel Julia Python event in Julia event in Python



Primary and flapper flow

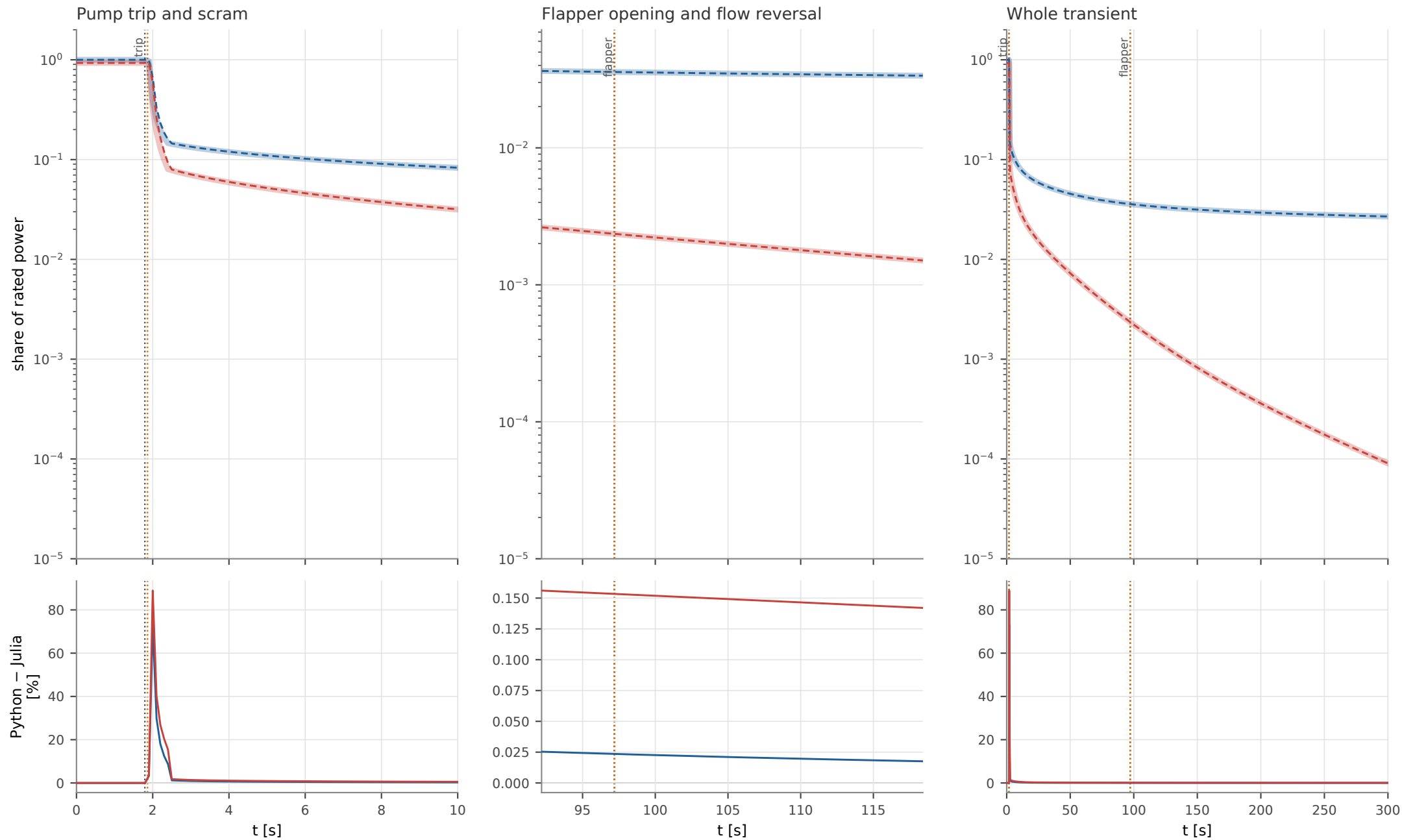
primary flapper Julia Python event in Julia event in Python



Power

The total is fission power plus decay heat. Below $1e-5$ of rated power the fission power is left off, and out of the difference.

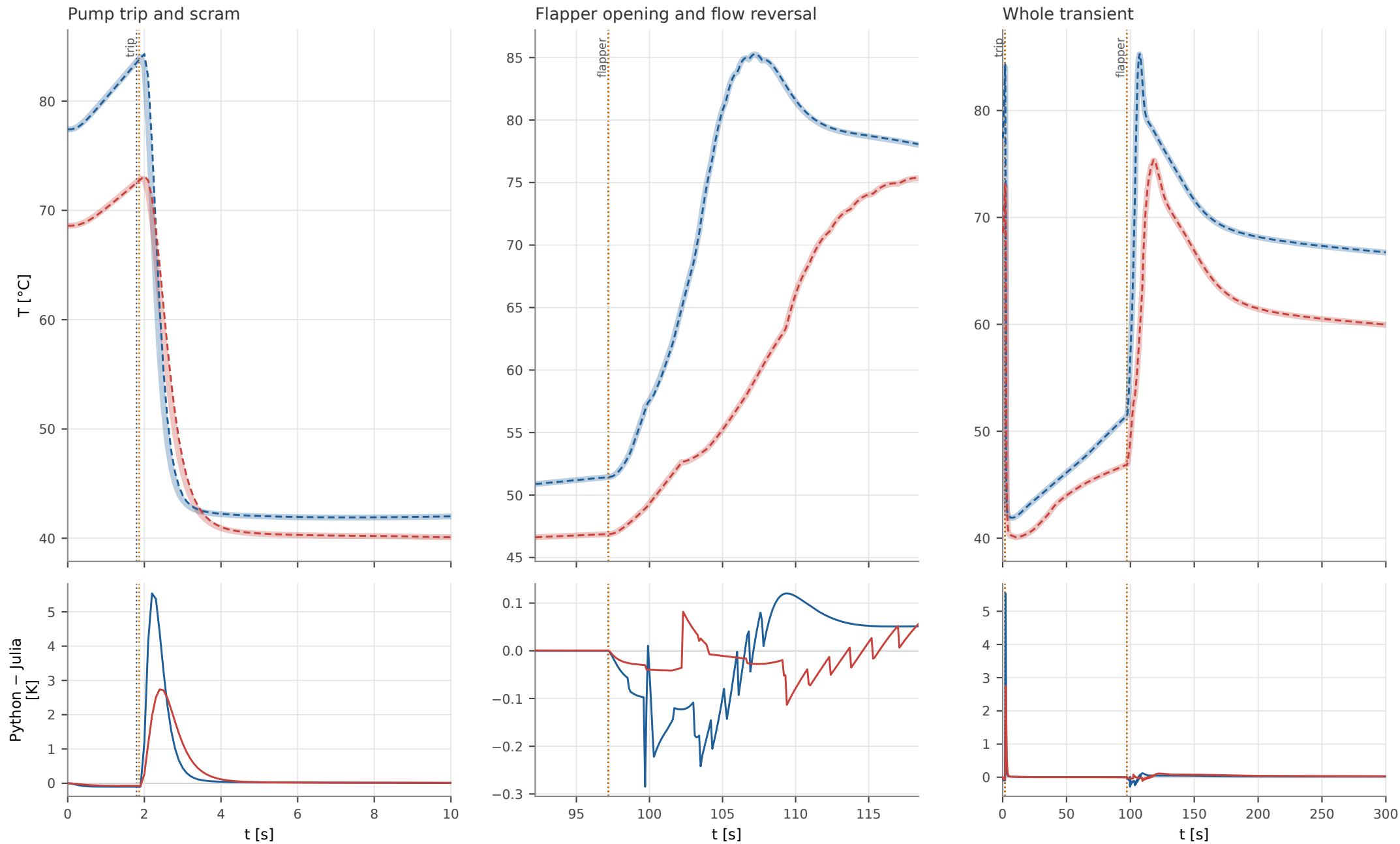
total, P fission, P_neutron Julia Python event in Julia event in Python



Hottest coolant

The hottest axial cell at each time.

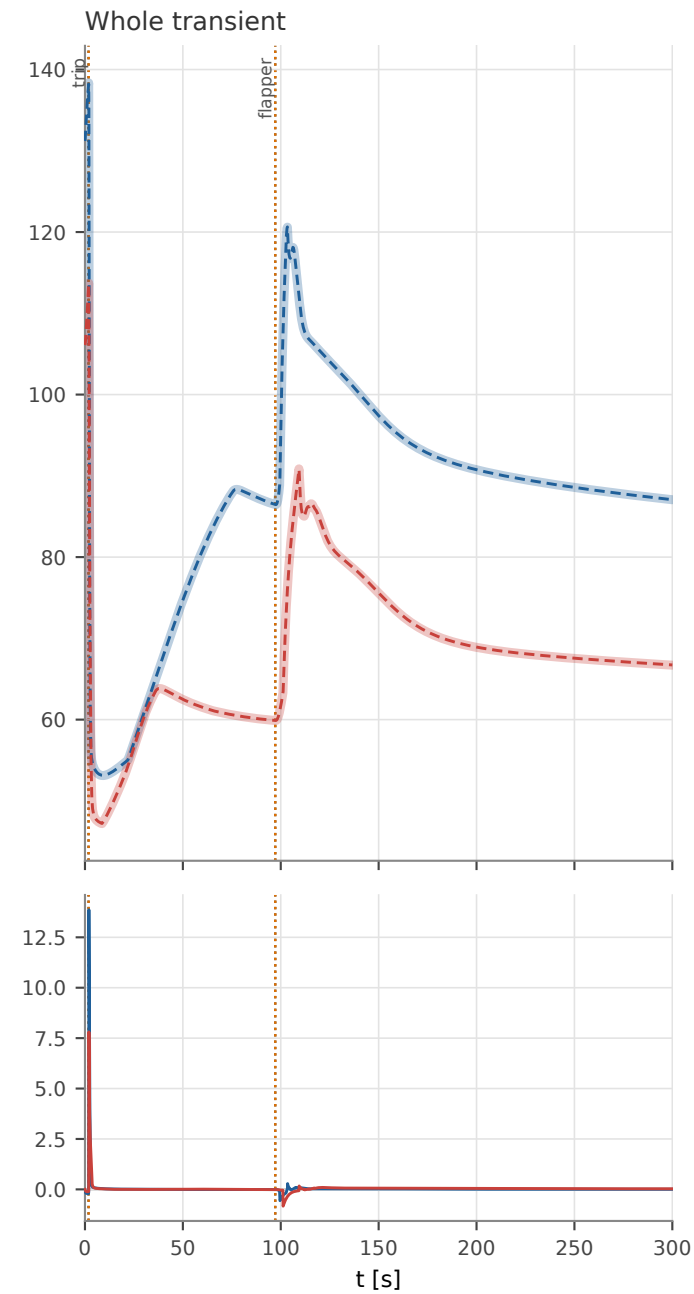
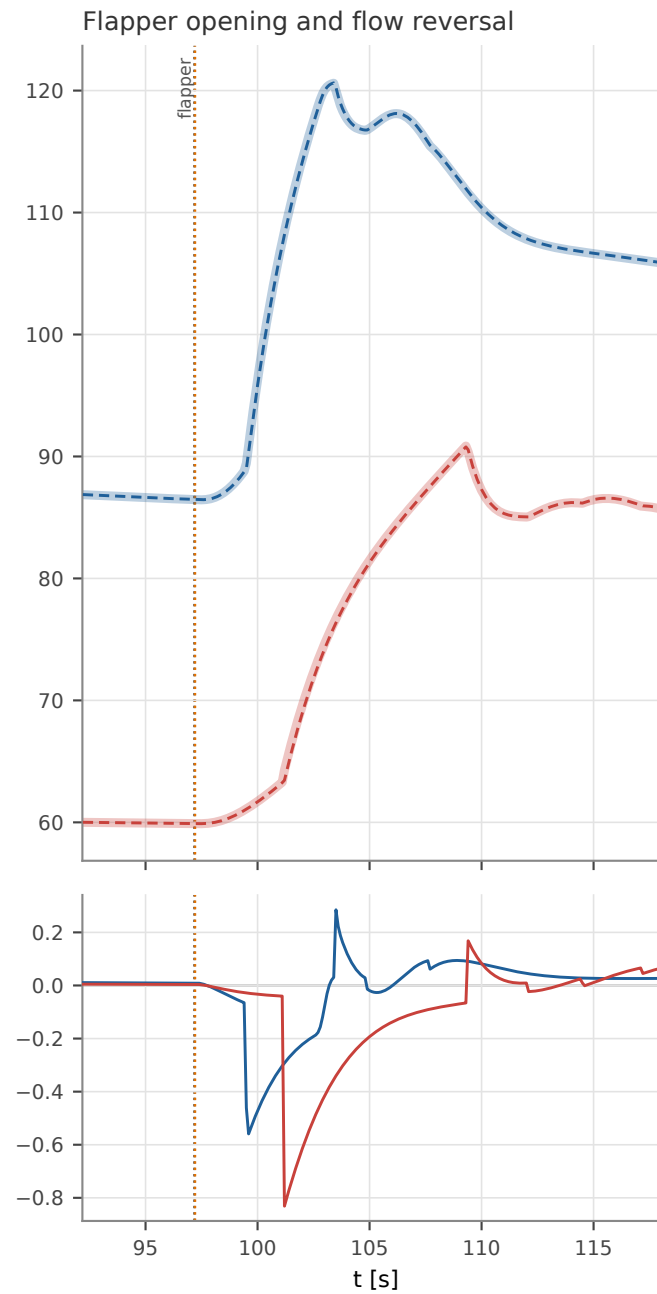
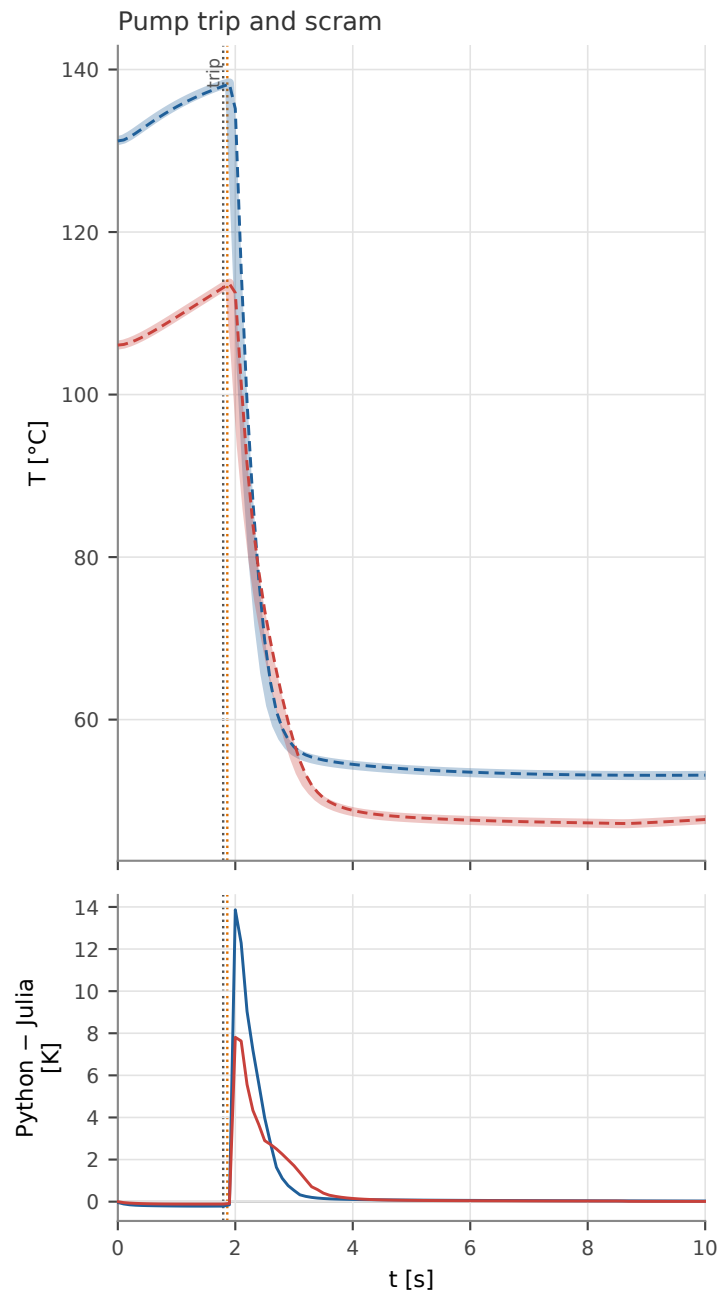
high channel low channel Julia Python event in Julia event in Python



Hottest wall

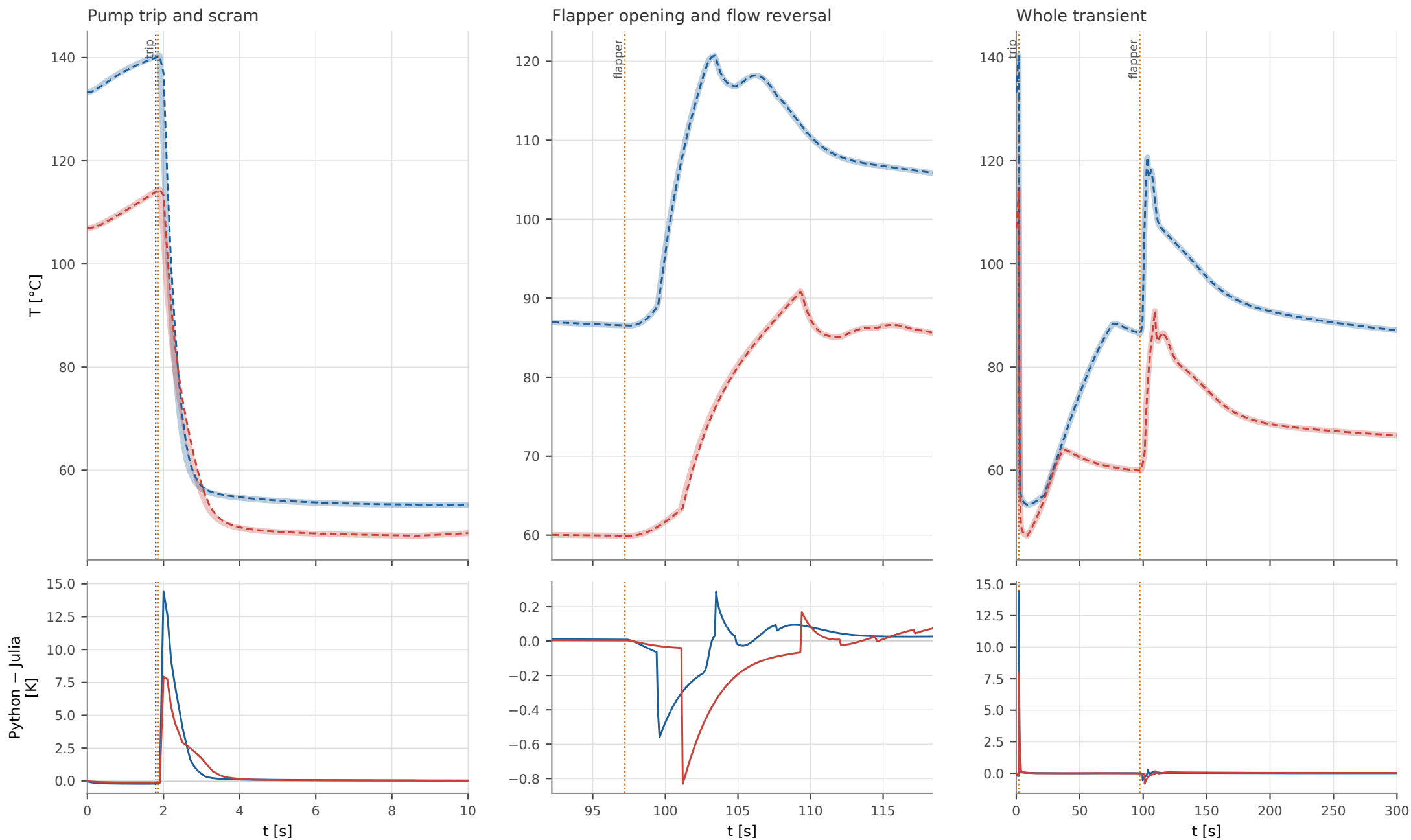
Plate surface facing the coolant, where boiling starts.

high channel low channel Julia Python event in Julia event in Python



Hottest plate

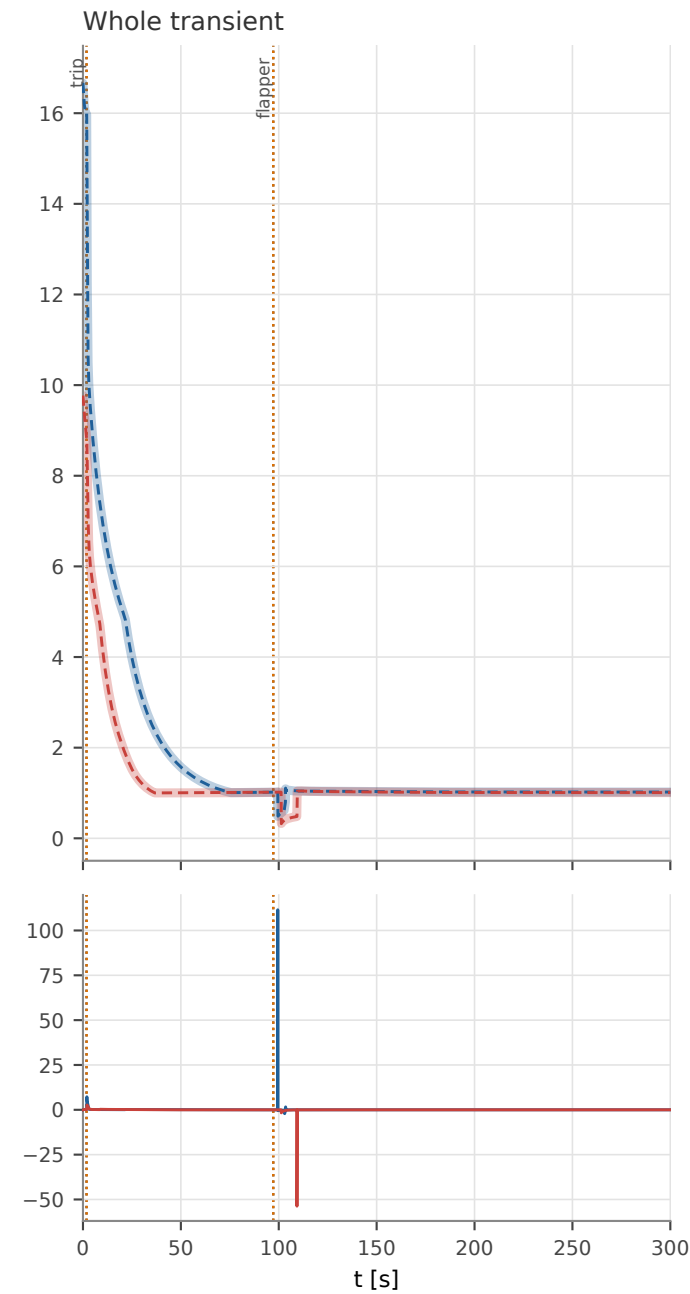
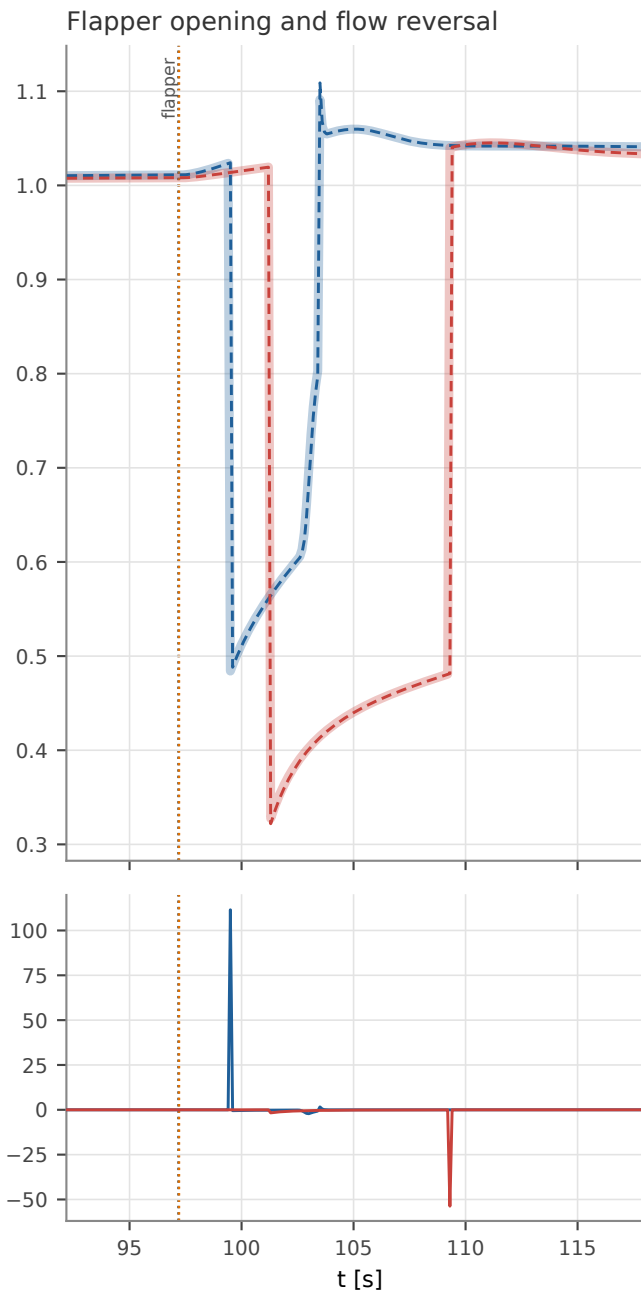
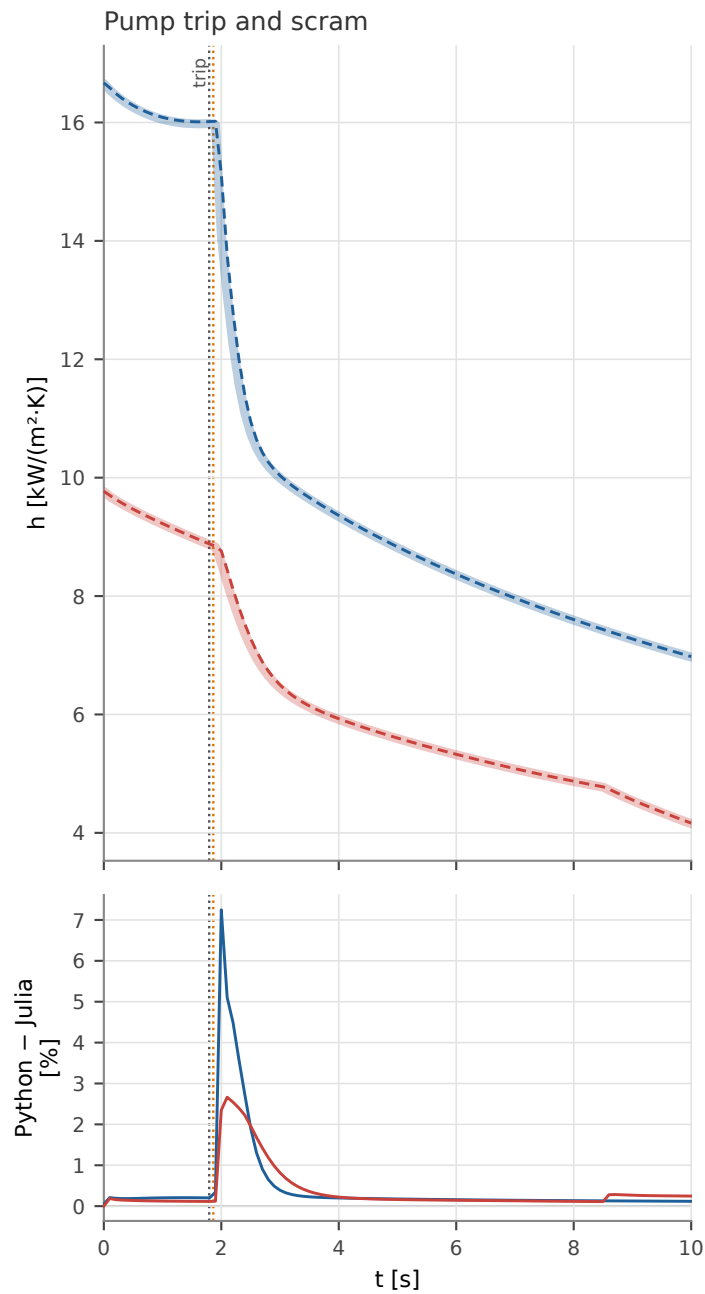
high channel low channel Julia Python event in Julia event in Python



Heat transfer coefficient

At the wall cell hottest at steady state, counted from the inlet: cell 6 of 10 in the high channel, cell 7 of 10 in the low channel.

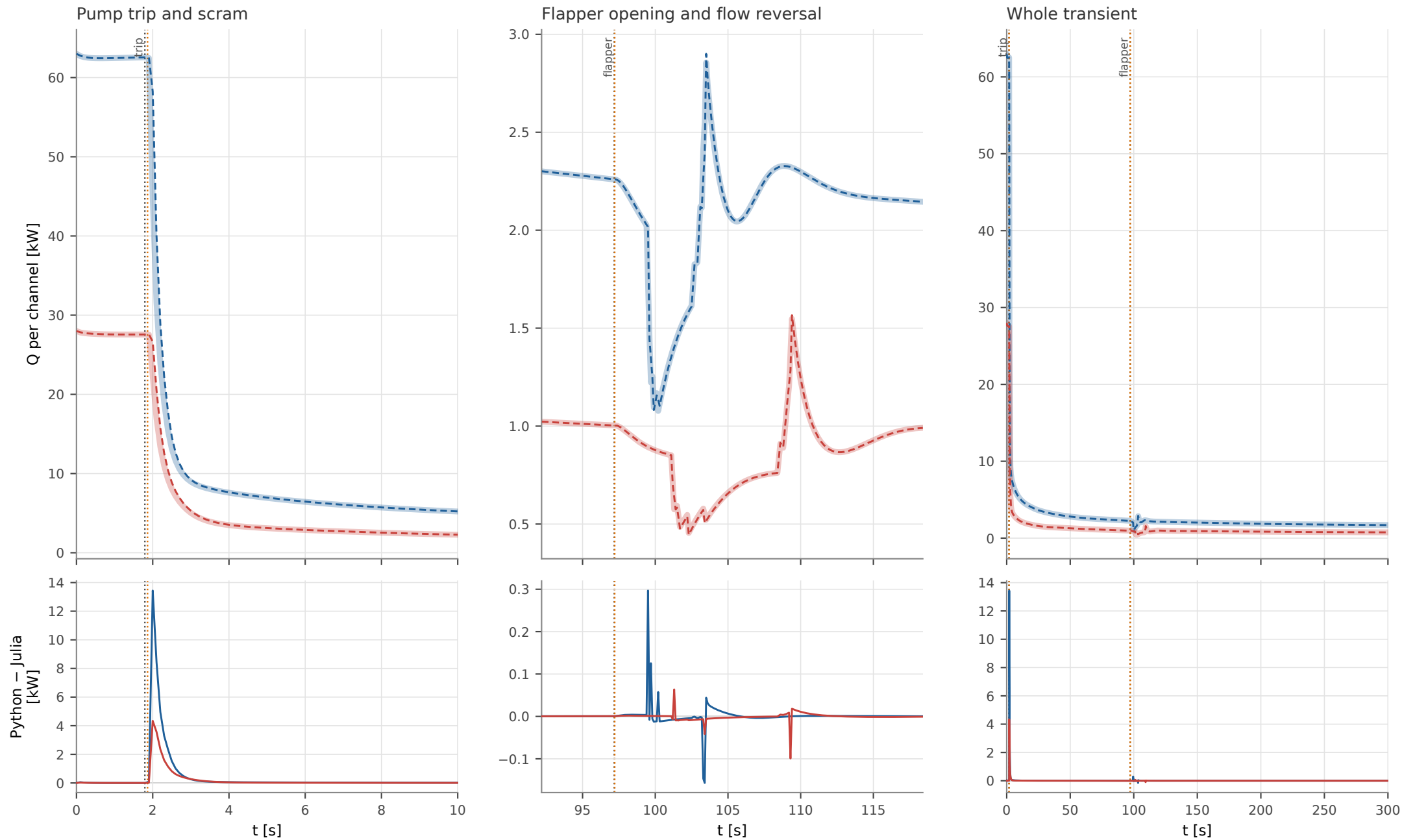
— high channel — low channel — Julia - - - Python event in Julia event in Python



Heat to the coolant

Through both faces of one channel.

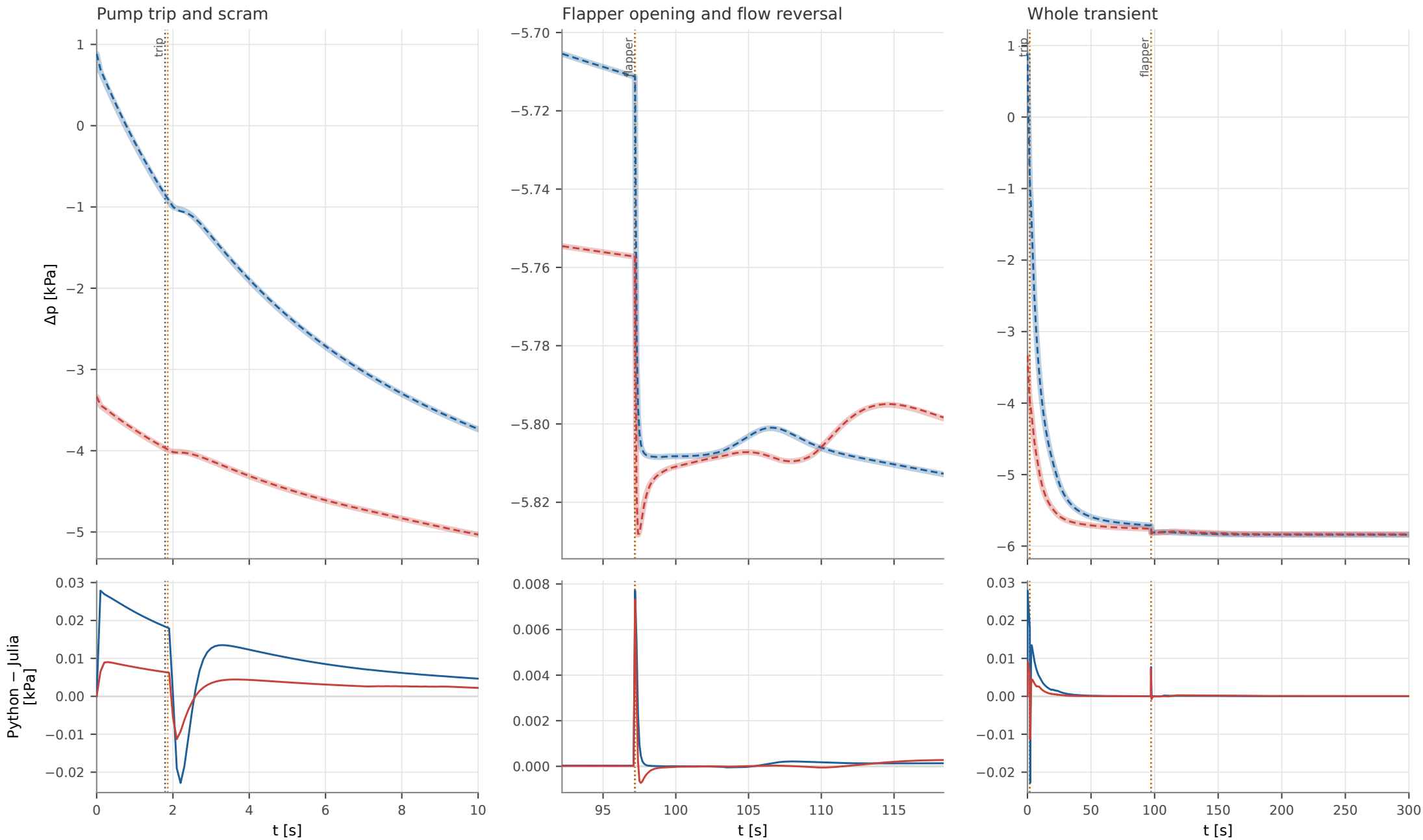
high channel low channel Julia Python event in Julia event in Python



Pressure drop, inlet to outlet

Friction, hydrostatic head and inertia. The core runs downward, so the head lowers the drop.

— high channel — low channel — Julia - - - Python event in Julia event in Python



Friction pressure drop

Darcy-Weisbach summed over the cells, with k_R scaling the Reynolds number of both friction branches.

— high channel — low channel — Julia ---- Python event in Julia event in Python

